A05 - Tataka Vana: The Corrupted Forest

Category: Reversing / Forensics | **Flag:** `PINAKA{tataka_vana_one_route_from_the_wreckage}`

Summary

Three undocumented binary files describe a trail network: a checkpoint graph, a damaged write-ahead log of later changes, and a log of shared beacons between eight drifting station clocks. I reversed all three formats, fitted each station's clock exactly from the beacons, replayed only the transactions that were truly committed, and ran an exact-rational Dijkstra. The key point is the replay rule. A transaction counts only if it is one unbroken, CRC-clean BEGIN..COMMIT run. Skipping torn frames inside a transaction looks harmless but commits garbage.

Recon

The three artifacts each start with an 8-byte magic: `PNKBCN01`, `PNKFRST1` and `PNKWAL01`. No schema is given, and the README says the flag isn't in any file. The validator wants the route, its exact cost, the recovered edge count, a SHA-256 digest of the recovered graph, and the per-station clock calibration.

```
A05_player ❯ ls -lgoh beacons.bin forest.store forest.wal && xxd beacons.bin | head -5 && xxd forest.store |
head -4 && xxd forest.wal | head -5
-rw-rw-r-- 1  404 Oct 30 07:05 beacons.bin
-rw-rw-r-- 1  19K Oct 30 07:05 forest.store
-rw-rw-r-- 1 3.4M Oct 30 07:05 forest.wal
00000000: 504e 4b42 434e 3031 0100 0800 0300 0000  PNKBCN01........
00000010: 9824 0c70 0000 0000 5081 9d3b 0000 0000  .$.p....P..;....
00000020: e558 9b77 0000 0100 4f90 cb3b 0000 0000  .X.w....O..;....
00000030: cc6c bc26 0000 0200 6b3f 603b 0000 0000  .l.&....k?`;....
00000040: c9f9 8f72 0000 0300 517c da3b 0000 0000  ...r....Q|.;....
00000000: 504e 4b46 5253 5431 0100 b100 0000 0002  PNKFRST1........
00000010: 0000 0000 b000 0800 0000 00bf 9ee2 0000  ................
00000020: 0000 1100 5369 6464 6861 7368 7261 6d61  ....Siddhashrama
00000030: 2d47 6174 6501 0001 0007 0052 6f77 616e  -Gate......Rowan
00000000: 504e 4b57 414c 3031 0100 ef65 0100 4f2f  PNKWAL01...e..O/
00000010: ef71 5746 524d 0f00 0100 0000 0000 0000  .qWFRM..........
00000020: 0100 0000 0101 00aa f732 8457 4652 4d2a  .........2.WFRM*
00000030: 0002 0000 0000 0000 0001 0000 0002 0101  ................
00000040: 0200 0004 00ab 0007 0029 9eb8 3b00 0000  .........)..;...

A05_player ❯
```

From the hex dumps and some trial parsing, the formats are:

- **beacons.bin**: magic, `u16` version, `u16` stations (8), `u32` beacons (3), then 24 records of `(u32 tag, u16 beacon, u16 station, u64 local_ts)`, then a CRC.
- **forest.store**: magic, `u16` version, `u32` nodes (177), `u32` edges (512), `u16` source (0), `u16` target (176), `u16` stations, `u16` reference station, `u32` CRC. Then the nodes as `(u16 id, u16 kind, u16 len, name)`, then 32-byte edges `(u32 eid, u16 src, u16 dst, u16 station_a, u64 ts_a, u16 station_b, u64 ts_b, u32 crc32)`.
- **forest.wal**: frames of `"WFRM" + u16 len + payload + u32 crc32`. The payload is `(u64 seq, u32 txn, u8 kind, body)`, where kind is 1 BEGIN, 2 EDGE op, 3 COMMIT or 4 ABORT. An EDGE body is `u8 op` (1 INSERT, 2 UPDATE, 3 DELETE) followed by the same fields as a store edge.

An edge's traversal cost is the elapsed time on the shared reference clock, `ref(station_b, ts_b) - ref(station_a, ts_a)`. Every cost therefore depends on the calibration, so the beacons have to be solved first.

Clocks and the damaged log

Each station's clock is linear in reference time: `local_s = alpha_s * ref + beta_s`. Station 0 is the reference, so the beacons' station-0 timestamps are the reference times. Two beacons give `alpha` and `beta` for every station as exact fractions, and the third beacon checks the fit with zero residual.

The WAL is the hard part. Scanning it for `WFRM` frames and checking each CRC gives this picture:

```
A05_player ❯ python wal_stats.py
WFRM frames: 85067 CRC-valid, 6558 CRC-broken (torn)
record kinds: BEGIN=26201 EDGE=41539 COMMIT=10760 ABORT=6567
transactions: 26202 total, 6567 aborted, 8875 never committed

lenient (skip bad frames)    : 10759 txns -> 4417 edges, 3017 inconsistencies
    txn 4214: INSERT of existing edge 415
    txn 4216: INSERT of existing edge 844
    txn 4234: INSERT of existing edge 1259
    txn 4276: INSERT of existing edge 629

strict (unbroken BEGIN..COMMIT): 4201 txns -> 933 edges, 0 inconsistencies

A05_player ❯
```

About 6.5k frames are torn, about 6.5k transactions abort, and nearly 9k never reach a COMMIT. The obvious approach is to drop the broken frames and keep every transaction with a valid BEGIN and COMMIT. That accepts 10,759 transactions and produces 3,017 contradictions, mostly INSERTs of edge IDs that already exist. That is the trap: a torn frame inside an open transaction means the transaction was half-written, even if a valid COMMIT follows later.

The correct rule is strict. A transaction is committed only if its records form one unbroken run, BEGIN, then EDGE ops, then COMMIT, with consecutive sequence numbers. A torn or foreign record inside the run leaves a gap in `seq`, which disqualifies it. That keeps 4,201 transactions, and replaying them over the checkpoint is perfectly self-consistent: no INSERT hits an existing edge and no UPDATE or DELETE misses one. That clean replay is what convinced me the rule was right.

Route

`solve.py` does everything from the raw files: calibrates the clocks, parses the checkpoint (checking each edge's CRC), replays the strictly committed transactions, converts every edge to an exact `Fraction` cost, runs Dijkstra from the store's source to its target, and writes `submission.json`.

```
A05_player ❯ python solve.py --offline
[+] clock calibration from beacons (max residual on 3rd beacon: 0)
    station 0: alpha=        1  beta=      0
    station 1: alpha=1003/1000  beta=  17961
    station 2: alpha=  249/250  beta= -13853
    station 3: alpha=  251/250  beta=  -4295
    station 4: alpha=1001/1000  beta=   -167
    station 5: alpha= 999/1000  beta=   1385
    station 6: alpha=1001/1000  beta= -21353
    station 7: alpha= 997/1000  beta=   9737
[+] checkpoint 512 edges + 4201 committed txns -> 933 edges, replay clean
[+] route  0 -> 4 -> 171 -> 176
    Siddhashrama-Gate -> Willow-4 -> Hollow-171 -> Tataka-Lair
[+] cost   3000
[+] digest 3752dbd9e8185e55afb27720bcce49a15d048286404f1de1980c1665d57042e3
[+] wrote submission.json

A05_player ❯
```

The core of it:

```python
cal, resid = forest.calibrate(forest.read_beacons())
st = forest.read_store()
recs, bad = forest.read_wal()
txns = forest.committed_txns(recs)              # strict BEGIN..COMMIT runs only
graph, problems = forest.replay(st["edges"], txns)
assert not problems

cost = forest.costs(graph, cal)                 # (ts - beta) / alpha on each end
digest = hashlib.sha256(";".join(
    f"{e}:{s}>{d}:{c.numerator}/{c.denominator}"
    for e, (s, d, c) in sorted(cost.items())).encode()).hexdigest()
route, total = dijkstra(cost, st["src"], st["tgt"])
```

The strict replay check in `forest.py`:

```python
clean = (kinds[0] == BEGIN and kinds[-1] == COMMIT
         and set(kinds[1:-1]) == {EDGE}
         and all(b[1] == a[1] + 1 for a, b in zip(rs, rs[1:])))
```

The answer is the route `0 -> 4 -> 171 -> 176` (Siddhashrama-Gate, Willow-4, Hollow-171, Tataka-Lair), with cost 3000 over a 933-edge graph.

Double-checking

Before submitting, `verify.py` re-checks the pieces independently. The beacon model is exact on all 24 observations, the replay has no contradictions, and every edge cost comes out a positive integer, which is a strong sign the calibration is right. It then removes each edge of the route in turn and re-runs Dijkstra to find the cheapest alternative. That costs 97000, so the 3000 route is the unique shortest one.

```
A05_player ❯ python verify.py
beacon model exact on all 3 beacons x 8 stations : True
WAL replay self-consistent                       : True
every recovered edge cost positive integer       : True
route cost recomputed                            : 3000
next-best route                                  : 0 -> 31 -> 68 -> 112 -> 137 -> 171 -> 176 cost 97000
unique shortest route                            : True

A05_player ❯
```

Flag

Posting `submission.json` to `/vana/passage` returns the flag. Running `python solve.py` without `--offline` does the same submission itself.




```
PINAKA{tataka_vana_one_route_from_the_wreckage}
```